

Technical Specification & Research Architecture

Blueprint: Latency-Compensated Edge-AI Visual Path Tracking for AGVs

Academic Institution: Vishwakarma Institute of Technology (VIT), Pune

Project Guide: Dr. Vijay Gaikwad

Hardware Target: NVIDIA Jetson Nano Developer Kit (4GB, FP16 TensorRT) &

Differential-Drive AGV Base

Target Publication Venues: IEEE Transactions on Industrial Informatics / IEEE Robotics and Automation Letters (RA-L)

1. Executive Summary & Research Novelty Pillars

This technical specification establishes the blueprint for an edge-optimized, latency-compensated vision navigation system for Autonomous Guided Vehicles (AGVs) operating in challenging industrial environments. Unlike standard vision pipelines that fail under non-Lambertian floor reflections or experience dynamic hunting due to inference lag, this framework introduces three distinct novelties:

- Specular-Invariant Chromaticity Transform (SICT):** A mathematical pre-filtering stage operating under the dichromatic reflection model to remove high-bay specular glare on epoxy/polyurethane floors before deep feature extraction.
 - Row-Anchor Neural Architecture (ShuffleNetV2 Backbone):** An ultra-lightweight structural line detection paradigm discretized over M horizontal anchor rows, running at sub-20 ms latency via NVIDIA TensorRT FP16 quantization.
 - Kinematic Latency Compensation via Dead-Reckoning Projection:** A predictive state estimator integrated into an Adaptive Pure Pursuit controller that compensates for total system transport delay τ (28.7 ms), eliminating closed-loop phase lag and steering oscillations.
 - CoppeliaSim (V-REP) Sim-to-Real Digital Twin:** Procedural visual domain randomization modeling specular gloss, abrasive tape loss, and variable lighting for zero-shot sim-to-real transferability.
-

2. Theoretical & Mathematical Formulations

2.1 Specular-Invariant Chromaticity Transform (SICT)

Given an RGB pixel vector $\mathbf{l}(u,v) = [R, G, B]^T$ with total intensity $S(u,v) = R + G + B$, the normalized chromaticity coordinates are: $r = \frac{R}{S + \epsilon}$, $g = \frac{G}{S + \epsilon}$, $b = \frac{B}{S + \epsilon}$ ($\epsilon = 10^{-5}$)

Specular reflections compress color toward the equal-energy illuminant point $(1/3, 1/3)$. The specular chromatic distance is formulated as: $\Phi(u,v) = \sqrt{\left(r(u,v) - \frac{1}{3}\right)^2 + \left(g(u,v) - \frac{1}{3}\right)^2}$

$I_{\text{filtered}}(u,v) = \begin{cases} I_{\text{ambient}}, & \text{if } S(u,v) \geq \Gamma_{\text{lum}} \text{ and } \Phi(u,v) \leq \Delta_{\text{spec}} \\ \alpha \cdot r(u,v) + \beta \cdot g(u,v), & \text{otherwise} \end{cases}$

Where Γ_{lum} is the saturation threshold, Δ_{spec} is the specular chromatic radius, and α, β tune sensitivity to safety markings.

2.2 Row-Anchor Path Classification & Structural Loss

Instead of dense pixel segmentation, the image ROI is sliced into M horizontal anchor rows $\mathcal{H} = \{h_1, \dots, h_M\}$, each discretized into K spatial gridding cells $\mathcal{C} = \{c_1, \dots, c_K\}$ plus a non-existence token c_{K+1} . The horizontal coordinate $\hat{w}_i$ along row h_i is derived via soft-argmax: $\hat{w}_i = \sum_{j=1}^K j \cdot \frac{\exp(\mathbf{P}_{i,j})}{\sum_{k=1}^{K+1} \exp(\mathbf{P}_{i,k})}$

The total training loss includes structural continuity constraints: $\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{focal}} + \lambda_{\text{smooth}} \sum_{i=1}^{M-1} |\hat{w}_{i+1} - \hat{w}_i| + \lambda_{\text{curv}} \sum_{i=1}^{M-2} |(\hat{w}_{i+2} - \hat{w}_{i+1}) - (\hat{w}_{i+1} - \hat{w}_i)|$

2.3 Non-Holonomic Kinematics & Latency Compensation

For a differential-drive robot with state $\mathbf{q} = [x, y, \theta]^T$, wheel radius r , and baseline B : $v = \frac{r}{2}(\omega_R + \omega_L)$, $\omega = \frac{r}{B}(\omega_R - \omega_L)$

Over total transport delay $\tau = t_{\text{act}} - t_{\text{cap}} = 28.7 \text{ ms}$, the dead-reckoning projected state is: $\hat{x}(t_{\text{act}}) \approx x(t_{\text{cap}}) + v(t_{\text{cap}}) \cos(\theta(t_{\text{cap}})) \cdot \tau$
 $\hat{y}(t_{\text{act}}) \approx y(t_{\text{cap}}) + v(t_{\text{cap}}) \sin(\theta(t_{\text{cap}})) \cdot \tau$
 $\hat{\theta}(t_{\text{act}}) = \theta(t_{\text{cap}}) + \omega(t_{\text{cap}}) \cdot \tau$

The look-ahead path target (x_{path}^*, y_{path}^*) is transformed into the projected future frame $\mathcal{R}(t_{act})$:
$$\begin{bmatrix} x_L^* & y_L^* & 1 \end{bmatrix} \mathcal{R}(t_{act}) = \begin{bmatrix} \cos(\Delta\theta) & \sin(\Delta\theta) & 0 \\ -\sin(\Delta\theta) & \cos(\Delta\theta) & 0 \\ 0 & 0 & 1 \end{bmatrix} \left(\begin{bmatrix} x_{path}^* & y_{path}^* & 1 \end{bmatrix} \mathcal{R}(t_{cap}) - \begin{bmatrix} v(t_{cap}) \cdot \tau & 0 & 0 \end{bmatrix} \right)$$
 Where $\Delta\theta = \omega(t_{cap}) \cdot \tau$. Steering curvature command: $\kappa_{cmd} = \frac{2 y_L^*}{L_d^2}$, $\omega_{cmd} = v(t_{act}) \cdot \kappa_{cmd}$

3. Physical Hardware Interfacing & Pinout Matrix

Subsystem	Hardware Component	Interface Protocol	Pin / Connection Details
Compute Core	NVIDIA Jetson Nano (4GB)	Board Power	5V/4A DC Barrel Jack (via synchronous Buck converter)
Visual Sensing	Sony IMX219 Camera Module	MIPI CSI-2	Jetson Nano 15-pin CSI connector (/dev/video0)
Motor Driving	TB6612FNG H-Bridge Driver	Hardware PWM & GPIO	PWM0 (Pin 32) -> PWMA; PWM1 (Pin 33) -> PWMB
Direction Logic	TB6612FNG Control Inputs	GPIO Digital Out	Pins 11, 13 (AIN1/AIN2); Pins 15, 16 (BIN1/BIN2)
Wheel Encoders	Dual Optical Quadrature	GPIO Interrupts	Pins 18, 22 (Left Ch A/B); Pins 29, 31 (Right Ch A/B)
Inertial Measurement	MPU-6050 6-DoF IMU	I2C Bus	Pin 3 (SDA), Pin 5 (SCL)

4. End-to-End Pipeline Performance Benchmarks

Stage	Sub-system / Execution Kernel	Latency (ms)	Target Frame Rate / Throughput
1	GStreamer CSI Frame Capture & ISP Debayering	4.8 ms	30–60 FPS
2	Specular-Invariant Chromaticity Transform (CUDA)	2.6 ms	Parallel GPU Execution
3	ShuffleNetV2 Row-Anchor Inference (TensorRT FP16)	16.4 ms	> 60 FPS Engine Capacity
4	Soft-Argmax Coordinate Extraction & IPM Spline Fit	3.7 ms	C++ ARM Execution
5	Dead-Reckoning Latency Compensation & Pure Pursuit	0.8 ms	ROS 2 Real-Time Control Loop
6	Linux Sysfs Hardware PWM Motor Output	0.4 ms	Hardware Timers
Total	Complete Closed-Loop Perception-to-Actua tion Cycle	28.7 ms	34.8 FPS Real-Time

5. Comparative Evaluation & Benchmark Baselines for Publication

To guarantee publishability, the experimental section must generate quantitative comparative tables against three primary baselines:

1. **Baseline 1:** Classical Heuristic Pipeline (Canny Edge Detection + Hough Line Transform + PID Controller).
 2. **Baseline 2:** Standard Deep Semantic Segmentation (MobileNetV2-UNet in TensorRT FP16 + Uncompensated Pure Pursuit).
 3. **Baseline 3:** Raw Row-Anchor Classification (UFLD without SICT Glare Filter + Uncompensated Pure Pursuit).
 4. **Proposed Architecture:** SICT Pre-filter + TensorRT FP16 Row-Anchor Classifier + Latency-Compensated Pure Pursuit.
-

6. CoppeliaSim Digital Twin & Sim-to-Real Domain Randomization

The simulation framework runs in CoppeliaSim using the Python ZeroMQ Remote API (`sim.getVisionSensorImage` and `sim.setJointTargetVelocity`).

Domain Randomization Parameters:

- **Floor Specular Exponent (η):** Uniform continuous distribution $\in [15, 120]$.
 - **Surface Diffuse Albedo (ρ_d):** Gaussian distribution $\mu=0.52, \sigma=0.12$.
 - **Lighting Intensity:** Log-normal distribution $\in [200, 1400]$ lux.
 - **Line Marking Abrasion:** Poisson gap distribution ($\lambda = 0.05$ breaks/m) with 0% – 45% material loss.
 - **Camera Pitch Offset ($\Delta\theta_p$):** Gaussian noise $\mu=0, \sigma=1.2^\circ$ (simulates chassis vibration).
-